

部署 GitHub Pages

🔧 第一阶段：教师端部署模板与测试

Step 1: 在 GitHub 创建教师模板仓库（Template Repo）

1. 登录 GitHub，创建一个全新的 Public 仓库（如 `haitec-student-template`）。
2. 在仓库根目录下创建以下基础文件结构：

Plain Text

```
1 .github/
2   └─ workflows/
3       └─ deploy.yml      <-- 自动编译与部署脚本
4 docs/
5   └─ index.md            <-- 个人主页/自我介绍
6   └─ week01.md          <-- 第一周文档
7   └─ images/            <-- 图片存放目录
8 mkdocs.yml              <-- MkDocs 核心配置文件
9 README.md               <-- 仓库说明
```

3. 在仓库的 **Settings** -> 勾选 **Template repository**（使其成为模板库）。

⚠️ 可能遇到的问题与解决方案：

- **问题：** `.github` 文件夹名称漏掉开头的 `.`，导致 GitHub Actions 无法识别。
- **解决：** 在网页端新建文件时直接输入 `.github/workflows/deploy.yml`，系统会自动创建正确的隐藏文件夹结构。

Step 2: 配置 `mkdocs.yml` 与自动部署脚本

1. 配置 `mkdocs.yml`：

YAML

```
1 site_name: 我的创客笔记
2 theme:
3   name: material
4   language: zh
5   palette:
6     primary: indigo
7     accent: blue
8   features:
9     - navigation.tabs
10    - content.code.copy
11
12 markdown_extensions:
13   - pymdownx.highlight
14   - pymdownx.superfences
15   - attr_list
16
17 plugins:
18   - search
```

2. 配置 `.github/workflows/deploy.yml` :

YAML

```
1 name: Deploy MkDocs to GitHub Pages
2
3 on:
4   push:
5     branches:
6       - main
7
8 permissions:
9   contents: write
10
11 jobs:
12   deploy:
13     runs-on: ubuntu-latest
14     steps:
15       - uses: actions/checkout@v4
16
17       - name: Configure Git Credentials
18         run: |
19           git config user.name github-actions
20           git config user.email github-actions@github.com
21
22       - name: Set up Python
23         uses: actions/setup-python@v5
24         with:
25           python-version: 3.x
26
27       - name: Install dependencies
28         run: |
29           pip install mkdocs-material
30
31       - name: Deploy
32         run: |
33           mkdocs gh-deploy --force
```

⚠ 可能遇到的问题与解决方案:

- **问题:** Action 运行报错 `Permission to ... denied to github-actions[bot]`。
- **解决:** 进入仓库 **Settings** -> **Actions** -> **General** -> **Workflow permissions**, 将权限改为 **Read and write permissions** 并保存。

Step 3: 激活 GitHub Pages 的 `gh-pages` 分支

1. 修改一次 `README.md` 并提交，触发第一次 Actions 自动构建。
2. 观察仓库顶部的 **Actions** 标签页，确保显示绿色的 .
3. 进入 **Settings** -> **Pages**，在 **Branch** 下拉菜单中选择自动生成的 `gh-pages` 分支，目录选 `/ (root)`，点击 **Save**。

⚠ 可能遇到的问题与解决方案：

- **问题：** Pages 里的 Branch 菜单中找不到 `gh-pages` 选项，只能看到 `main` 或 `none`。
 - **解决：** 说明 Actions 脚本尚未成功运行过。先检查 `.github/workflows/deploy.yml` 的语法与路径，在 Actions 运行成功（显示绿勾）后重新刷新 Pages 页面即可出现。
-

🎓 第二阶段：学生建站（使用网页端快速建库）

Step 4: 学生通过模板创建自己的专属仓库

1. 学生注册并登录自己的 GitHub 账号。
 2. 打开教师提供的模板仓库页面，点击绿色的 **Use this template -> Create a new repository**。
 3. 仓库名填写为 `my-fab-notes`，勾选 **Public**，点击 **Create repository**。
-

Step 5: 学生开启个人的 GitHub Pages 服务

1. 学生进入自己的新仓库，点击 **Settings** -> **Pages**。
2. 将 **Branch** 设置为 `gh-pages`，点击 **Save**。
3. 等待 1~2 分钟，页面上方会生成学生专属的个人网站链接（如 `https://<student-username>.github.io/my-fab-notes/`）。

⚠ 可能遇到的问题与解决方案：

- **问题：** 访问网页显示 404 或直接跳转到原始 README 文件。
 - **解决：** 检查 Pages 设置里绑定的分支是否误选成了 `main`，必须选 `gh-pages` 才能展示编译后的 MkDocs 网页。
-

💻 第三阶段：学生本地环境搭建与密钥绑定 (VS Code + SSH)

Step 6: 本地软件安装与 Git 身份初始化

1. 学生电脑安装 **Git** 与 **VS Code**。
2. 打开 VS Code 的终端 (Terminal)，运行以下命令设置 Git 身份：

Bash

```
1 git config --global user.name "学生GitHub用户名"  
2 git config --global user.email "学生GitHub注册邮箱"
```

Step 7: 配置 SSH 密钥（一劳永逸解决网络报错）

为避免 HTTPS 传输导致的 443 超时或 SSL 重置问题，统一采用 SSH 连接：

1. 在 VS Code 终端生成 SSH 密钥：

Bash

```
1 ssh-keygen -t ed25519 -C "学生GitHub注册邮箱"
```

(连续按三下回车，使用默认设置)

2. 打印并复制公钥：

Bash

```
1 cat ~/.ssh/id_ed25519.pub
```

(复制输出的以 `ssh-ed25519` 开头的整行内容)

3. 登录 GitHub -> 右上角头像 -> **Settings** -> **SSH and GPG keys** -> **New SSH key** -> 粘贴并保存。
4. 在终端验证：

Bash

```
1 ssh -T git@github.com
```

(出现 `Hi <username>! You've successfully authenticated...` 即表示绑定成功)

⚠️ 可能遇到的问题与解决方案：

- **问题：** 终端提示 `Permission denied (publickey)`。
- **解决：** 说明复制公钥时漏掉或多出了字符，或者没有粘贴到 GitHub 账号对应的 Settings 中，需重新复制 `cat ~/.ssh/id_ed25519.pub` 的完整内容并重新添加。

Step 8: 克隆仓库到本地

1. 学生在 GitHub 自己的仓库页面，点击绿色的 **Code** 按钮，切换到 **SSH** 标签，复制形如 `git@github.com:<username>/my-fab-notes.git` 的链接。
2. 在 VS Code 终端运行：

Bash

```
1 git clone git@github.com:<username>/my-fab-notes.git
```

3. 用 VS Code 打开刚刚克隆下来的文件夹。

📝 第四阶段：日常更新与文档维护流程

Step 9: 编写 Markdown 文档与插入本地图片

1. 学生在 `docs/` 目录下编写或新建 Markdown 文件（如 `docs/week01.md`）。
2. **插入本地图片规范：**
 - 将图片经过 WebP 压缩后放入 `docs/images/` 目录下。
 - 在 Markdown 中使用相对路径调用：

Markdown

```
1 ![PCB走线](images/pcb_design.webp)
```

⚠️ 可能遇到的问题与解决方案：

- **问题：** 网页上线后图片破裂无法显示。
- **解决：** 切勿在路径前加上 `../docs/`。在 MkDocs 中，`docs/` 是根目录，在 `docs/week01.md` 引用图片时直接写 `images/图片名.webp` 即可。

Step 10: 提交更新与同步部署

编写完成后，在 VS Code 终端依次执行三步命令（或使用左侧源代码管理面板操作）：

Bash

```
1 # 1. 暂存所有修改
2 git add .
3
4 # 2. 提交修改并说明原因
5 git commit -m "docs: add week01 log"
6
7 # 3. 推送到 GitHub
8 git push origin main
```

推送成功后，后台 GitHub Actions 会自动进行编译，约 1~2 分钟后，学生的个人网站即会自动更新！

⚠ 可能遇到的问题与解决方案：

- **问题：**Push时报 `fatal: Exiting because of an unresolved conflict` 或 `error: Committing is not possible because you have unmerged files`。
- **解决：**说明网页端或远程有修改与本地未同步。先运行 `git pull origin main`，然后在 VS Code 的左侧 Git 面板中解决高亮的冲突块，再重新进行 `git add . -> git commit -> git push`。